

Лабораторная работа

Управление процессами и потоками в QNX Neutrino

Методические указания

1 Цель работы

Получение навыков практической работы в операционной системе реального времени QNX Neutrino. Изучение основных способов создания и порождения процессов и потоков в операционной системе QNX.

2 Процессы и потоки

2.1 Атрибуты процесса

Использование процессов, работающих в изолированных друг от друга адресных пространствах, для выполнения различных задач - отличительная черта всех операционных систем семейства **QNX**. Такой подход в первую очередь позволяет повысить надежность разрабатываемых систем.

Процесс является контейнером потоков. Собственно, всю реальную работу делают именно потоки, поэтому любой процесс состоит как минимум из одного потока.

Процесс имеет ряд атрибутов:

- идентификатор процесса (**Process ID**, **PID**) - уникальный номер, присваиваемый процессу при его порождении операционной системой;
- идентификатор родительского процесса (**Parent PID**, **PPID**) - **PID** процесса, породившего данный процесс, т. е. выполнившего запрос к операционной системе для создания данного процесса;
- реальные идентификаторы владельца и группы (**User ID**, **UID** и **Group ID**, **GID**) - номер, позволяющий механизмам защиты информации от несанкционированного доступа (НСД) определять, какому пользователю принадлежит процесс и к какой группе пользователей принадлежит этот пользователь. Эти идентификаторы присваиваются при регистрации пользователя в системе или командному интерпретатору (**login shell**), если выполнялась командно-строковая регистрация через утилиту **login**, или графической оболочке **Photon**, если регистрация выполнялась в графическом режиме через утилиту **phlogin**. Процессы, запускаемые пользователем, наследуют **UID** и **GID** той программы, из которой они запускаются (т. е. *родительского процесса*);
- эффективные идентификаторы владельца и группы (**Effective UID**, **EUID** и **Effective GID**, **EGID**) - предназначены для повышения гибкости механизмов защиты информации от НСД. Пользователь при наличии соответствующих полномочий может в ходе работы менять эффективные идентификаторы. При этом реальные идентификаторы не меняются. Механизмы, реализующие дискреционную защиту информации от НСД, для проверки прав доступа используют эффективные идентификаторы;
- текущий рабочий каталог - путь (разделенный слешами список каталогов), который будет автоматически добавляться к относительным именам файлов. Выводится на экран командой **pwd**;
- управляющий терминал - терминал, с которым связаны потоки ввода, вывода и ошибок;
- маска создания файлов (**umask**) - атрибуты доступа, которые будут заданы для файла, созданного процессом;
- значение приоритета;
- дисциплина диспетчеризации;
- использование ресурсов процессора (статистика по времени выполнения программы) - включает: время выполнения программы в прикладном контексте (**user time** - время выполнения инструкций, написанных программистом), время

выполнения в контексте ядра (*system time* - время выполнения инструкций ядра по запросу программы, т. е. системных вызовов), суммарное время выполнения всех дочерних процессов в прикладном контексте, суммарное время выполнения всех дочерних процессов в контексте ядра.

2.2 Атрибуты потока

Хотя потоки внутри процесса разделяют его адресное пространство, каждый поток имеет также "частные" данные. В некоторых случаях эти частные данные защищены ядром (например *tid* или *Thread ID* - идентификатор потока), в то время как другие приватные данные находятся в незащищенном адресном пространстве процесса (например, каждый поток имеет стек для собственных нужд). Перечислим некоторые из наиболее важных ресурсов, относящихся к потоку:

tid - каждый поток идентифицируется целым *thread ID* (Идентификатором потока), начинающимся с единицы. *tid* - это уникальное число характеризующее поток процесса.

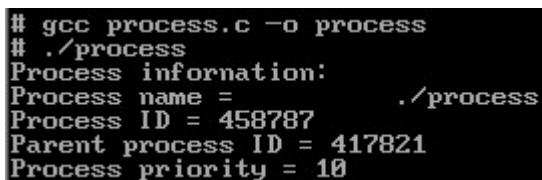
priority - каждый поток имеет приоритет, который помогает определить когда он запускается. Приоритет потока наследуется от начального приоритета родительского потока, но приоритет может быть изменен, в зависимости от политики планирования, точное изменение делает сам поток, или сообщения посылаемые потоку.

В *QNX Neutrino* процессы не имеют приоритетов, их имеют лишь потоки.

Пример 1.

Проиллюстрируем, каким образом программа может получить информацию о значении своих атрибутов (файл *process.c*):

```
#include<stdlib.h>
#include<stdio.h>
#include<sys/resource.h>
int main(int argc, char **argv, char **env)
{
    struct rusage r_usage;
    printf("\nProcess Information:\n");
    printf("Process name=\t\t%s\n", argv[0]);
    printf("Process ID (PID)=\t< %d >\n", getpid());
    printf("Parent's process ID (PPID)=\t< %d >\n", getppid());
    printf("Process priority= \t< %d >\n", getprio(0));
    return EXIT_SUCCESS;
}
```



```
# gcc process.c -o process
# ./process
Process information:
Process name =          ./process
Process ID = 458787
Parent process ID = 417821
Process priority = 10
```

Жизненный цикл процесса можно разделить на четыре этапа:

- ☐ создание процесса;
- ☐ загрузка образа процесса;
- ☐ выполнение процесса;
- ☐ завершение процесса.

3 Порождение процесса

"Предком" всех процессов является процесс *Администратор процессов*

(процесс **procnto**), идентификатор **PID** которого равен 1. Остальные процессы порождаются в результате вызова соответствующей функции другим процессом, именуемым *родительским*. Таких функций несколько:

- **system()** - порождает **shell** для выполнения командной строки, указанной в качестве аргумента.
- **fork()** - создает дочерний процесс путем "клонирования" родительского процесса;

Примечание

Долгое время в **UNIX**-подобных системах новые процессы порождались путем вызова сначала функции **fork()**, а затем из дочернего процесса-клона - функции семейства **exec()**.

Рекомендуется использовать **fork()** только в однопоточных программах.

- семейство функций **exec()** - отличаются набором аргументов функций, заменяют образ вызвавшего процесса указанным исполняемым файлом;

□ **vfork()** ("виртуальный **fork()**") - используется как "облегченная" альтернатива вызову **fork()**. В отличие от стандартной функции **fork()**, она не выполняет реального копирования данных, а просто блокирует родительский процесс, пока дочерний не вызовет **exec()**;

- семейство функций **spawn*()** - сразу порождает дочерний процесс, загрузив указанный исполняемый файл. Наиболее эффективный способ порождения процессов в **QNX Neutrino**;

3.1 Использование командного интерпретатора

Самый простой способ - запустить из программного кода дочернюю копию командного интерпретатора, которому затем передать команду запуска процесса. Для этого используется вызов:

```
int system(const char * command);
```

где **command** - текстовая строка, содержащая команду, которую предполагается выполнить ровно в том виде, в котором мы вводим ее командному интерпретатору с консоли.

Примечание

Функция имеет еще одну специфическую форму вызова, когда в качестве **command** задается **null**. По коду возврата это позволяет выяснить, присутствует ли (и доступен ли) командный интерпретатор в системе (возвращается 0, если интерпретатор доступен).

На время выполнения вызова **system()** вызывающий процесс приостанавливается. После завершения порожденного процесса функция возвращает код завершения вновь созданной копии интерпретатора (или **-1**, если сам интерпретатор не может быть выполнен), то есть младшие 8 бит возвращаемого значения содержат код завершения выполняемого процесса. Возврат вызова **system()** может анализироваться макросом **WEXITSTATUS()**, определенным в файле **<sys/wait.h>**. **Пример 2:**

```

#include<stdlib.h>
#include<stdio.h>
#include<sys/resource.h>
#include<sys/wait.h>
int main(int argc, char **argv, char **env)
{
    int rc = system("ls");
    if(rc == -1) printf("shell could not be run");
    else
        printf("result of running command is \t<%d>\n ",
WEXITSTATUS(rc));
    return EXIT_SUCCESS,
}

```

Вызов `system()` является не только простым, но и очень наглядным, делающим код легко читаемым. Программисты часто относятся к нему с пренебрежением, отмечая множество его недостатков. Однако в относительно простых случаях это может быть оптимальным решением, а недостатки не так и существенны:

- Используя копию командного интерпретатора, вызов `system()` может инициировать процесс, исполняющий и бинарную программу, и скрипт на языке самого командного интерпретатора (`shell`), а также программный код на интерпретирующих языках, таких как *Perl*, *Tcl/Tk* и др. Многие из рассматриваемых ниже «чисто программных» способов могут загружать и исполнять только бинарный исполняемый код (по крайней мере, без использования ими весьма громоздких искусственных и альтернативных возможностей).

- Остановка родительского процесса в ожидании завершения порожденного также легко разрешается: просто запускайте дочерний процесс из отдельного потока.

3.2 Клонирование процесса

`fork()`

Вызов `fork()` создает клон (полную копию) вызывающего процесса в точке вызова. Вызов `fork()` является одной из самых базовых конструкций всего *UNIX*-программирования. Его толкованию посвящено столько страниц в литературе, сколько не уделено никакому другому элементу *API*. Действие вызова `fork()` следующее:

- Порождается дочерний процесс, которому системой присваивается новое уникальное значение *PID*.

- Дочерний процесс получает собственные копии файловых дескрипторов, открытых в родительском процессе в точке выполнения `fork()`. Каждый дескриптор ссылается на тот же файл, который соответствует аналогичному дескриптору родителя. Блокировки файлов (*locks*), установленные в родительском процессе, наследуются дочерним процессом.

- Для дочернего процесса его значения *tms_utime*, *tms_stime*, *tms_cutime* и *tms_cstime* устанавливаются в значение ноль. Выдержки (*alarms*) для этих таймеров, установленные к этому времени в родительском процессе, в дочернем процессе очищаются.

Сигнальные маски (подробнее об этом будет рассказано ниже) для дочернего процесса инициализируются пустыми сигнальными наборами (независимо от сигнальных масок, установленных родительским процессом).

Если вызов функции завершился неудачно, функция возвращает `-1` и устанавливает `errno`.

Для одного вызова `fork()` одновременно имеют место два кода возврата в

двух различных копиях (но в текстуально едином коде!): в копии кода, соответствующей дочернему процессу, возвращается 0, а в копии кода родителя - *PID* успешно порожденного дочернего процесса. Это и позволяет разграничить в едином программном коде фрагменты, которые после точки ветвления надлежит выполнять в родительском процессе и которые относятся к дочернему процессу. Типичный шаблон кода, использующего `fork()`, выглядит примерно так:

```
pid_t pid = fork();
if(pid == -1) perror("fork"); exit(EXIT_FAILURE);
if(pid == 0)
{
    // . . этот код выполняется в дочернем процессе ...
    exit(EXIT_SUCCESS);
};
if(pid > 0)
{
    // . этот код выполняется в родительском процессе ...
    do
    {
        // ожидание завершения порожденного процесса
        wpid = waitpid(pid, &status, 0);
    };
    while(WIFEXITED(status) == 0);
    exit(WEXITSTATUS(status));
};
```

Эта схема порождения процесса, его клонирование, настолько широко употребляется, особенно при построении самых разнообразных серверов, что для нее была создана специальная техника, построенная на вызове `fork()`. Во всех многозадачных ОС обязательно присутствует та или иная техника программного создания нового процесса, однако не во всех существует техника именно клонирования, то есть создания полного дубликата порождающего процесса.

Все порождаемые процессы наследуют весь набор дескрипторов, доступных родительскому процессу. Лучшее, что могут сделать процессы (как родительский, так и дочерний), - немедленно после вызова `fork()` (и это хорошая практика в общем случае) тщательно закрыть все унаследованные дескрипторы, не имеющие отношения к их работе.

Пример 3.

Посмотрим, как применяют функцию `fork()` на примере файла `fork.c`. В этом примере видно, что функция `fork()` возвращает целое число, которое в родительском процессе равно нулю, а в дочернем - идентификатору процесса:

```

#include<stdlib.h>
#include<stdio.h>
#include<sys/resource.h>
int main(int argc, char **argv, char **env)
{
    pid_t pid;
    char *Prefix;
    Prefix=(char *) malloc (sizeof(char));
    pid=fork();
    if (pid == 0) sprintf (Prefix, "Child:");
    else sprintf (Prefix, "Parent:");

    printf("%s Process name=%s\n", Prefix, argv[0]);
    printf("%s Process ID=%d\n", Prefix, getpid());
    printf ("%s Parent's process ID=%d\n", Prefix, getppid
());
    return EXIT_SUCCESS;
}

```

```

# gcc fork.c -o fork
# ./fork
Parent: Process name = ./fork
Parent: Process ID = 864301
Parent: Parent's process ID = 491549
Child: Process name = ./fork
Child: Process ID = 868398
Child: Parent's process ID = 864301

```

vfork()

В отличие от `fork()`, этот вызов, впервые введенный в [BSD UNIX](#), делает разделяемым для дочернего процесса адресное пространство родителя. Родительский процесс приостанавливается до тех пор, пока порожденный процесс не выполнит `exec()` (загружая новый программный код дочернего процесса) или не завершится с помощью `exit()` или аналогичных средств.

Функция `vfork()` может быть реализована на аппаратных платформах с физической моделью памяти (без виртуализации памяти), а `fork()` - не может (или реализуется с большими накладными расходами), так как требует создания абсолютной копии области адресного пространства, что в физической модели повлечет сложную динамическую модификацию адресных полей кода. Тем не менее (при некоторых кажущихся достоинствах) в [BSD](#) также подчеркивалось, что `vfork()` таит в себе серьезную потенциальную опасность, поскольку позволяет одному процессу использовать или даже модифицировать адресное пространство другого, то есть фактически нарушает парадигму защищенных адресных пространств.

3.3 Запуск нового программного кода

Наконец, рассмотрим запуск на выполнение программного кода, отличного от родительского процесса (образ которого содержится в отдельном исполняемом файле) в качестве дочернего процесса. Для этой цели в [QNX](#) существуют две группы функций: `exec()` (их всего 8: `execl()`, `execle()`, `execclp()`, `execclpe()`, `execv()`, `execve()`, `execvp()`, `execvpe()`) и `spawn()` (их 10: `spawn()`, `spawnl()`, `spawnle()`, `spawnlp()`, `spawnlpe()`, `spawnp()`, `spawnv()`, `spawnve()`, `spawnvp()`, `spawnvpe()`).

Это множество форм записи отличается синтаксисом, который определяет формат списка аргументов командной строки, полученного в качестве параметров функции `main()`, передаваемых программе, а также некоторыми другими дополнительными деталями. Суффиксы в именах функций обозначают следующее:

□ **l** - список аргументов определяется через список параметров, заданных непосредственно в самом вызове. Этот список завершается нулевым аргументом **NULL**;

□ **e** - окружение для процесса указывается посредством определения массива переменных окружения;

□ **p** - относительный путь поиска: если не указан полный путь к файлу программы (то есть имя файла не содержит разделителей «/»), для его поиска используется переменная окружения **PATH**;

□ **v** - список аргументов определяется через указатель на массив аргументов.

Большинство форм функции **exec()** являются **POSIX**-совместимыми. Функции семейства **exec()** подменяют исполняемый код текущего процесса (не изменяя его идентификатор **PID**, права доступа, внешние ресурсы процесса, а также находящийся в том же адресном пространстве) исполняемым кодом из другого файла. Поэтому используются эти вызовы непосредственно после **fork()** или **vfork()** для замены копии вызывающего процесса новым (это классическая **UNIX**-технология использования).

Пример 4.

На примере **vfork.c** проиллюстрируем порождение нового процесса с помощью комбинации вызовов **vfork()** и **exec()**:

```
#include<stdlib.h>
#include<stdio.h>
#include<process.h>
#include<sys/resource.h>
int main(int argc, char **argv, char **env)
{
    pid_t pid;
    pid=vfork();
    if (pid == 0)
    {
        execlp("process", "process", NULL);
        perror("Child");
        exit( EXIT_FAILURE);
    }
    waitpid(0, NULL, 0);
    printf("Parent's process ID=%d\n", getpid());
    printf("Parent's parent's process ID=%d\n", getppid());
    return EXIT_SUCCESS;
}
```

```
# gcc vfork.c -o vfork
# ./vfork
Process information:
Process name =          process
Process ID = 1007658
Parent process ID = 1003555
Process priority = 10
Parent's process ID = 1003555
Parent's parent's process ID = 491549
```

3.4 Порождение нового процесса с помощью **spawn()**

Большая часть форм функции **spawn()** представляет собой специфическое расширение **QNX**. Более того, даже для тех функций группы **spawn()**, которые часто называют **POSIX**-совместимыми, техническая документация **QNX** определяет степень совместимости примерно в таких терминах: «...функция

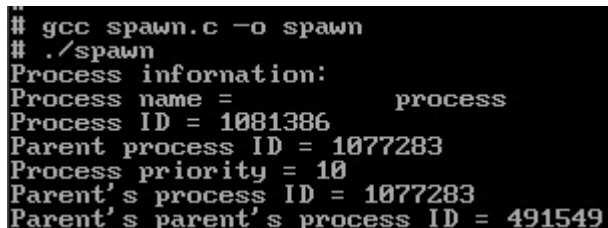
`spawn()` является функцией **QNX Neutrino** (основанной на **POSIX 1003.1d** черновом стандарте)».

Функции семейства `spawn()` порождают новый процесс (с новым идентификатором **PID** и в новом адресном пространстве). Все формы вызовов `spawn()` после подготовительной работы (иногда очень значительной) в конечном итоге ретранслируются в вызов базовой формы `spawn()`, который последним действием своего выполнения и посылает сообщение **procnto** (менеджер процессов **QNX**, «территориально» объединенный с микроядром системы в одном файле).

Пример 5.

В файле `spawn.c` представлен пример наиболее простого и быстрого способа порождения нового процесса:

```
#include<stdlib.h>
#include<stdio.h>
#include<process.h>
int main(int argc, char **argv, char **env)
{
    spawnl(P_WAIT, "process", "process", NULL );
    printf("Parent's process ID=%d\n", getpid());
    printf("Parent's parent's process ID=%d\n", getppid());
    return EXIT_SUCCESS;
}
```



```
# gcc spawn.c -o spawn
# ./spawn
Process information:
Process name =          process
Process ID = 1081386
Parent process ID = 1077283
Process priority = 10
Parent's process ID = 1077283
Parent's parent's process ID = 491549
```

Даже из этого очень краткого обзора вызова `spawn()` становятся очевидными некоторые вещи:

- Эта форма универсальна (самодостаточна), она позволяет обеспечить весь спектр разнообразных форм порождения нового процесса
- Она же и самая громоздкая форма, тяжеловесная для практического кодирования, поэтому в реальных текстах в большинстве случаев вы вместо нее встретите ее конкретизации: `spawnl()`, `spawnle()`, `spawnlp()`, `spawnlpe()`, `spawnp()`, `spawnv()`, `spawnve()`, `spawnvp()`, `spawnvpe()`. Функционально они эквивалентны `spawn()`.
- Хотя вызов `spawn()` и упоминается в описаниях как **POSIX**-совместимый, в **QNX** он существенно расширен и модифицирован и поэтому в лучшем случае может квалифицироваться как «выполненный по мотивам» **POSIX**.

3.5 Завершение процесса

Процесс завершается, если программа выполняет вызов `exit()` или выполнение просто доходит до точки завершения функции `main()`, будь то с явным указанием оператора `return` или без него. Это естественный, внутренний (из программного кода самого процесса) путь завершения. Число, передаваемое при этом в качестве аргумента функции `exit()` или с оператором `return`, называют *кодом возврата* программы. Если программа запускалась из командной строки в интерактивном режиме, то ее код возврата можно посмотреть в служебной переменной окружения `?`:

```
echo $?
```

Другой путь - посылка процессу извне (из другого процесса) сигнала, реакцией на который (предопределенной или установленной) является завершение процесса (подробнее о сигналах и реакциях см. ниже). В противовес естественному завершению такое принудительное завершение извне (по крайней мере, в отношении потоков) называют **отменой**, и именно этим термином мы будем пользоваться далее, чтобы отчетливо отмечать, о каком варианте завершения идет речь. (Такая же терминология будет использоваться нами и относительно завершения потока.)

Завершить процесс можно, пошлав ему сигнал с помощью утилиты **slay** или **kill** в командной строке, или из программы с помощью таких функций, как **kill()**, **sigqueue()** и др.

Завершение процесса выполняется в две стадии.

- На первой стадии происходит "физическое" уничтожение процесса, т. е. закрываются открытые файлы, освобождается оперативная память и т. д. Эта стадия осуществляется потоком-завершителем Администратора процессов, выполняющимся от имени уничтожаемого процесса.

- На второй стадии уничтожается структура данных о процессе, хранящаяся в операционной системе. Эта стадия выполняется Администратором процессов внутри самого себя.

Примечание

Для того чтобы в родительском процессе прочесть код возврата дочернего процесса, можно воспользоваться какой-либо из функций семейства **wait*()**.

Обратите внимание, что из всего этого семейства **POSIX**-функциями являются только **wait()** и **waitpid()**.

После окончания первой стадии уничтожения процесса и до того, как родительский процесс вызвал функцию **wait*()**, Администратор процессов не заканчивает вторую стадию уничтожения, а завершаемый процесс находится в состоянии, называемом "зомби". То есть "зомби" - это такой процесс, который физически уже не существует, но его код завершения все еще хранится в Администраторе процессов.

Примечание

Потоки процесса-"зомби" находятся в состоянии "**DEAD**-блокирован". Кстати, а что если функция **wait()** вызвана до завершения дочернего процесса? Тогда родительский процесс (вернее, вызвавший эту функцию поток) станет **WAIT**-блокированным и будет оставаться в таком состоянии до завершения дочернего процесса.

При порождении дочернего процесса с помощью функции **spawn*()** можно указать флаг **P_NOWAITO**. В этом случае порожденный процесс никогда не сможет стать процессом-"зомби", т. к. Администратор процессов не будет хранить код возврата. Соответственно, нельзя будет использовать функции **wait*()** для получения кода возврата.

Существует функция **atexit()**, позволяющая зарегистрировать для процесса функции-деструкторы, т. е. функции, вызываемые при нормальном завершении процесса непосредственно перед завершением. Таких деструкторов можно зарегистрировать 32, причем вызываться они будут в порядке **LIFO**.

3.6 Получение информации о процессах и потоках

Для быстрого получения информации о текущем состоянии процессов и потоков в Neutrino используют несколько утилит:

- **ps** - основная **POSIX**-утилита для мониторинга процессов. Она включена в **QNX** как для совместимости **POSIX**, так и для удобства администраторов, недавно работающих в **QNX**;

□ **pidin** - эта утилита появилась в **QNX** только с 6-й версии и предназначена для получения детальной информации о потоках.

3.7 "Посмертная" диагностика процессов

"Посмертная" диагностика процессов основана на использовании программы **dumper**, сохраняющей на диске образ "аварийного" процесса - так называемый **core-файл** (иногда именуемый **core**-образом процесса). Эту программу можно использовать и для немедленного получения образа какого-либо работающего процесса. Если программа **dumper** запущена системой, то по умолчанию она сохраняет **core**-образы в каталоге **/var/dumps** с именами в формате **имя_процесса.core**.

Полученный образ процесса можно анализировать двумя способами: либо утилитой **coreinfo**, сразу выдающей информацию о **core**-образе, либо с помощью **GNU**-отладчика **gdb**:

```
gdb имя_программы имя_программы.core
```

Анализ **core**-файла в отладчике можно сделать более приятным. С помощью **IDE**, разумеется.

Примечание

Многие поставщики программного обеспечения требуют, чтобы при обращении в их службы технической поддержки клиенты предоставляли соответствующие **core**-образы.

4 Потоки

Техника потоков вводит новую парадигму программирования вместо уже ставших традиционными **UNIX**-методов. Это обстоятельство часто недооценивается. Например, использование **pthread_create()** вместо **fork()** может на порядки повысить скорость реакций, особенно в ОС с отсутствием механизмов **COW** (*copy on write*) при создании дубликатов физических страниц **RAM** сегментов данных (таких как **QNX**, хотя механизмы **COW** вряд ли вообще применимы в ОС реального времени). Другой пример: использование множественных потоков вместо ожиданий на множестве дескрипторов в операторе **select()**.

Поток можно понимать как любой автономный последовательный (линейный) набор команд процессора. Источником этого линейного кода для потока могут служить:

- бинарный исполняемый файл, на основе которого системой или вызовом группы **spawn()** запускается новый процесс и создается его **главный** поток;
- дубликат кода главного потока процесса родителя при клонировании процессов вызовом **fork()** (тоже относительно **главного** потока);
- участок кода, оформленный функцией специального типа (**void* () (void*)**); это общий случай при создании второго и всех последующих потоков процесса (при создании многопоточных процессов) вызовом **pthread_create()**. Такую функцию мы будем называть функцией потока. Это наиболее интересный для нас случай.

В первых двух вариантах мы имеем неявное создание (главного) потока и, как следствие, порождение нового процесса. В последнем случае - явное создание потока, которое в литературе, собственно, и именуется «созданием потока». Хотя сущность происходящего относительно исполняющегося потока во всех случаях все же остается неизменной.

Кроме последовательности команд к потоку нужно отнести и те локальные данные, с которыми работает функция потока, то есть собственный стек потока.

Во время приостановки системой выполнения (диспетчеризации) кода текущего потока должна обеспечиваться возможность сохранения текущих значений регистров (включая регистры *FPU*, сегментные регистры) и, возможно, другой специфической информации. Текущее значение этого набора данных, относящихся к выполнению текущего потока, называется контекстом потока. Контекст потока, кроме того, обеспечивает связь потока с его экземпляром собственных данных, о чем мы детально поговорим чуть позже. Детальная структура и объем данных, составляющих контекст потока, определяются не только самой ОС, но и типом процессорной архитектуры, на которой она выполняется (для многоплатформенных ОС, к которым принадлежит и *QNX*).

В принципе считается, что время переключения контекстов потоков в пределах одного процесса и время переключения контекстов процессов могут заметно отличаться, особенно для платформ с управлением виртуальной памятью. Однако удобства реализации и стремление к однородности могут перевесить соблазн разработчиков ОС использовать это различие, что мы вскоре и увидим в отношении *QNX*.

Идентификатором потока (значимым только внутри одного процесса!) является *TID* (*Thread ID*), присваиваемый потоку при его создании вызовом *pthread_create()*. *TID* позволяет процессу (а также системе в рамках процесса) однозначно идентифицировать каждый поток. Нумерация *TID* в *QNX* начинается с 1 (это всегда главный поток процесса, порожденный *main()*) и последовательно возрастает по мере создания потоков (до 32767).

Еще одним важнейшим атрибутом потока является приоритет его выполнения. Для каждого из уровней приоритетов, обслуживаемых системой (в *QNX* 6.2.1 таких уровней 64, в *QNX* 6.3 - 256), поддерживается циклическая очередь потоков, готовых к исполнению (на деле большая часть из таких очередей оказывается пустой). Все политики диспетчеризации работают только с потоками из одной такой очереди: очереди потоков наивысшего из присутствующих в системе приоритетов. Если в системе выполняется поток высокого приоритета, то ни один поток более низкого приоритета не получит управление до тех пор, пока поток высокого приоритета не будет переведен в заблокированное состояние в ожидании некоторого события.

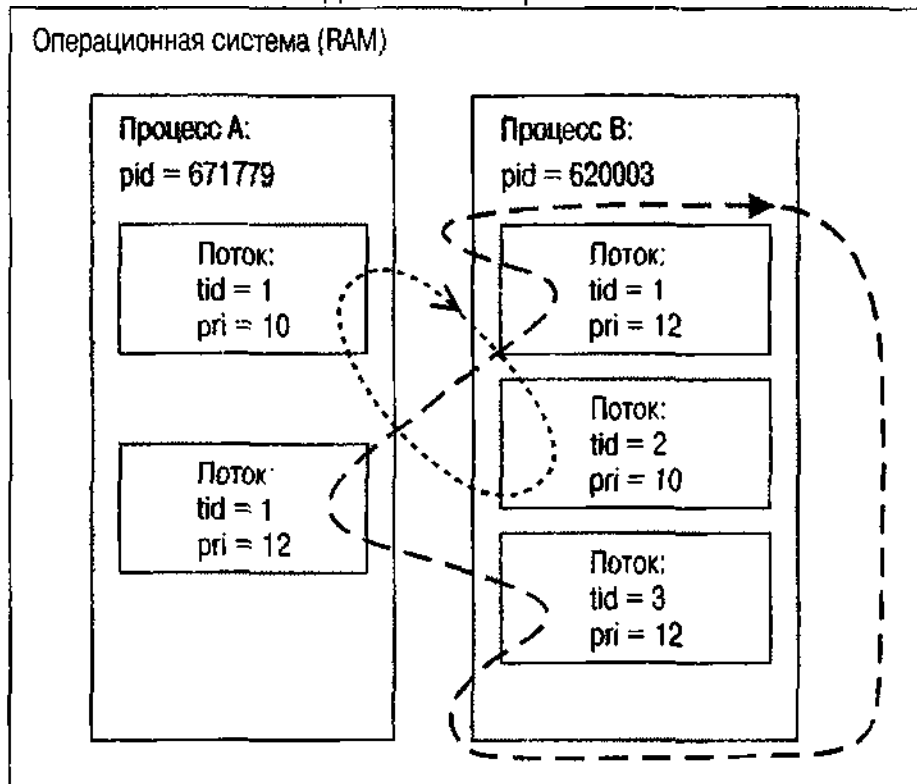


Рис. 2.3. Диспетчеризация потоков с различными приоритетами

На рис. 2.3 представлены два процесса, каждый из которых создает внутри себя несколько потоков, но на этот раз различных приоритетов (10 и 12). Жирной пунктирной линией показан порядок, в котором потоки высокого приоритета (12) объединены в циклическую очередь диспетчеризации. Это активная очередь диспетчеризации (наивысшего приоритета). Тонкой линией показан порядок потоков в другой очереди (приоритета 10). До тех пор пока все потоки активной очереди не окажутся в силу каких-либо обстоятельств в заблокированном состоянии, ни один из потоков очереди приоритета 10 не получит ни единого кванта времени.

4.1 Создание нового потока

Создание нового потока в программном коде осуществляет вызов:

```
int pthread_create(pthread_t* thread,  
const pthread_attr_t* attr, void* (*start_routine)(void*),  
void* arg);
```

где **thread** - **NULL** или указатель переменной типа **pthread_t**, значение которой будет загружено идентификатором созданного потока после успешного выполнения функции. Далее это значение (это и есть **TID**) может использоваться по тексту программы для идентификации созданного потока.

attr - **NULL** или указатель структуры типа **pthread_attr_t**. Если это значение **NULL**, то созданный поток будет иметь набор параметров, устанавливаемых по умолчанию. Если нет, то поток будет создан с параметрами, установленными в структуре **attr**. Модификация полей **attr** после создания потока (то есть после вызова функции) не оказывает никакого эффекта на параметры потока, и вообще говоря, структура **attr** может быть уничтожена сразу же после вызова **pthread_create()**. Документация предостерегает от прямой манипуляции значениями полей этой структуры, предлагая использовать для этого функции **pthread_attr_init()** и **pthread_attr_set_*()**.

start_routine - функция типа **void* ()(void*)**, уже упоминавшаяся выше как функция потока; это тот код, который будет фактически выполняться в качестве отдельного потока. Если выполнение этой функции завершается по **return**, то происходит нормальное завершение потока с вызовом **pthread_exit()**, использующим значение, возвращаемое **start_routine** в качестве **статуса завершения**. (Исключением является поток, связанный с **main()**; он при завершении выполняет вызов **exit()**.)

arg - указатель на блок данных, передаваемых **start**, **routine** в качестве входного параметра. Этот параметр подробно рассмотрен далее.

Чаще всего (однако совершенно необязательно) функция потока **start_routine** представляет собой бесконечный цикл, в котором выполняются некоторые действия с выходом из цикла в том случае, когда нужно завершить выполнение и уничтожить созданный поток. Выглядит это следующим образом:

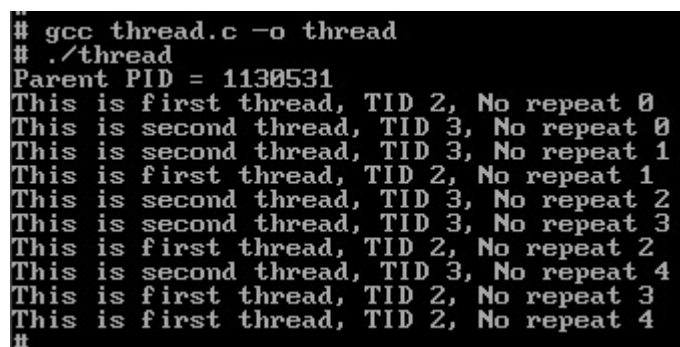
```
// функция потока:  
Void* ThreadProc(void* data)  
{  
while(true)  
{  
// ... выполняется работа ...  
if() break;  
// после этого поток нам уже не нужен!  
}  
return NULL;  
}
```

После успешного создания нового потока он начинает функционировать «параллельно» с породившим его потоком и другими потоками процесса (если

быть совсем точными, то со всеми прочими потоками, существующими в системе, так как в **QNX** существует только одна стратегия диспетчеризации потоков **PTHREAD_SCOPE_SYSTEM**, и существует она глобально, на уровне всей системы). При этом после точки выполнения **pthread_create()** невозможно предсказать, какой поток получит управление: породивший, порожденный или вообще произвольный поток из другого процесса. Это важно учитывать при передаче новому потоку данных и других операциях начальной инициализации параметров внутри созданного потока.

Пример 6.

```
#include <stdio.h>
#include <pthread.h>
#include <sys/neutrino.h>
pthread_t thread_id1;
pthread_t thread_id2;
void * long_thread1(void *notused)
{
    int n;
    for(n=0;n<5;n++)
    {
        printf("This is first thread, TID %d - N repeat %d \n",
thread_id1, n );
        sleep(2);
    }
}
void * long_thread2(void *notused)
{
    int m;
    for(m=0; m<5; m++)
    {
        printf("This is second thread, TID %d - N repeat %d \n",
thread_id2 , m );
        sleep(1);
    }
}
int main(void)
{
    printf("PID %d \n",getpid());
    pthread_create(&thread_id1, NULL, long_thread1, NULL);
    pthread_create(&thread_id2, NULL, long_thread2, NULL);
    sleep(40);
    return(1);
}
```



```
# gcc thread.c -o thread
# ./thread
Parent PID = 1130531
This is first thread, TID 2, No repeat 0
This is second thread, TID 3, No repeat 0
This is second thread, TID 3, No repeat 1
This is first thread, TID 2, No repeat 1
This is second thread, TID 3, No repeat 2
This is second thread, TID 3, No repeat 3
This is first thread, TID 2, No repeat 2
This is second thread, TID 3, No repeat 4
This is first thread, TID 2, No repeat 3
This is first thread, TID 2, No repeat 4
#
```

В отличие от создаваемых параллельных процессов, рассмотренных ранее, все потоки, создаваемые в рамках одного процесса, разделяют единое адресное пространство процесса, и поэтому все переменные процесса, находящиеся в области видимости любого потока, доступны этому потоку.

4.2 Завершение потока

Как и в случае обсуждавшегося ранее завершения процесса, для потоков мы будем отчетливо различать случаи:

- «естественного» завершения выполнения потока из кода самого потока;
- завершения потока извне, из кода другого потока или по сигналу. Для этого действия, в отличие от «естественного» завершения, будем использовать другой термин - отмена.

Завершение потока происходит при достижении функцией потока своего естественного конца и выполнения оператора **return** (явно или неявно) или выполнения потоком вызова:

```
void pthread_exit(void* value_ptr)
```

где **value_ptr** - указатель на результат выполнения потока.

При выполнении **pthread_exit()** поток завершается. Если этот поток принадлежит к категории ожидаемых, он может возвратить результат своей работы другому потоку, ожидающему его завершения на вызове **pthread_join()** (только один поток может получить результат завершения). Если же этот поток отсоединенный, то по его завершении все системные ресурсы, задействованные потоком, освобождаются немедленно.

Перед завершением потока будут выполнены все завершающие процедуры, помещенные в стек завершения, а также деструкторы собственных данных потока, о которых мы говорили ранее. Для последнего потока процесса вызов **pthread_exit()** эквивалентен **exit()**.

4.3 Возврат результата потока

Выше отмечено, что вызов **pthread_exit()**, завершающий ожидаемый поток, может передать результат выполнения потока. То же действие может быть выполнено и оператором **return** потоковой функции, которая из прототипа ее определения должна возвращать значение типа **void***.

В обоих случаях результат может иметь сколь угодно сложный структурированный тип; никакая типизация результата не предусматривается (тип **void***). Важно, чтобы код, ожидающий результата на вызове **pthread_join()**, понимал его так же, как и функция потока, возвращающая этот результат.

Другим условием является то, что переменная «результат» должна существовать к моменту вызова **pthread_join()**, то есть вполне возможно, что уже далеко после завершения самой функции ожидаемого потока. Этому условию не удовлетворяют, например, любые локальные для функции потока объекты, размещаемые в стеке.

5 Содержание отчета по лабораторной работе

Отчет по лабораторной работе должен содержать следующие сведения:
цель работы;
примеры выполнения команд;
пояснительные комментарии к командам.

6 Приложение

```
spawnl()
```

Создает дочерний процесс с заданным списком аргументов

Краткий обзор:

```
#include <process.h>
```

```
int spawnl( int mode,
```

```
const char * path,  
const char * arg0,  
const char * arg1...,  
const char * argn,  
NULL );
```

Аргументы:

mode

Как вы хотите загрузить дочерний процесс и как вы хотите, чтобы родительская программа вела себя после запуска дочерней программы:

P_WAIT — загружает дочернюю программу в доступную память, выполняет ее и заставляет родительскую программу возобновить выполнение после завершения дочернего процесса.

P_NOWAIT — выполняет родительскую программу одновременно с новым дочерним процессом.

P_NOWAITO — выполнение родительской программы одновременно с новым дочерним процессом. Вы не можете использовать **wait()** для получения кода выхода.

P_OVERLAY — замените родительскую программу дочерней программой в памяти и запустите дочернюю. Возврат к родительской программе не производится. Это эквивалентно вызову соответствующей функции **exec*()**.

path

Полный путь к исполняемому файлу.

arg0, argn, NULL

Аргументы, которые вы хотите передать новому процессу. Вы должны завершить список аргументом, равным **NULL**.

Библиотека:

libc

Используйте опцию **-l** с для **gcc**, чтобы связать с этой библиотекой. Обычно эта библиотека включается автоматически.

Описание:

Функция **spawnl()** создает и выполняет новый дочерний процесс, имя которого указано в **path** со списком аргументов, завершающимся нулевым значением в **arg0 ... argn**. Эта функция вызывает **spawnve()**.

execlp()

Выполнить файл

Краткий обзор:

```
#include <process.h>
```

```
int execlp( const char * file,  
const char * arg0,  
const char * arg1,  
...  
const char * argn,  
NULL );
```

Аргументы:

file

Используется для создания пути, идентифицирующего новый файл образа процесса. Если аргумент **file** содержит косую черту, аргумент **file** используется в качестве пути к файлу. В противном случае префикс пути для этого файла получается путем поиска каталогов, переданных в качестве переменной окружения **PATH**.

arg0, ..., argn

Указатели на символьные строки, заканчивающиеся нулем. Эти строки составляют список аргументов, доступный для нового образа процесса. Завершает список указатель, заканчивающийся нулем. Аргумент `arg0` должен указывать на имя файла, связанное с процессом.

Библиотека:

`libc`

Используйте опцию `-l` с для `gcc`, чтобы связать с этой библиотекой. Обычно эта библиотека включается автоматически.

Описание:

Функция `execvp()` заменяет текущий образ процесса на новый образ процесса, указанный в файле. Новый образ создается из обычного исполняемого файла, называемого файлом нового образа процесса. Возврат не производится, поскольку образ вызывающего процесса заменяется новым образом процесса.